Edinburgh Napier UNIVERSITY

Dashboards

Dr Peter Chapman

SET09123 – Interactive Data Visualisation

Lecture Summary

- Dashboards with CSS Flexbox
- Dashboards with CSS Grids

Edinburgh Napier UNIVERSITY

CSS Layouts

No CSS

- Before CSS was a thing (1996) your standard web page was either:
  - Not structured, elements came one after the other, in one big column
  - Structured using HTML tables, which isn’t great for reusability

Early CSS

- With CSS, the style property *`display`* was proposed
  - *`inline`*: no line break between items, ignores width and height properties
  - *`block`*: items have a width and height, but with a forced line break
  - *`inline-block`*: no line break, respects the width and height properties

- Then came floats: elements can be *floating*, left or right of the page
  - Originally designed to have images alongside text
  - Hacked to allow *`div`* elements to be freely positioned on the page

Bootstrap

- In 2011, Twitter open-sourced their CSS library for creating consistent web pages
- Bootstrap proposed a simple grid system to allow for an easy layout of pages
  - 12 columns were available, as many rows as necessry
  - Elements could span several columns and rows
  - Bootstrap took care of the page’s responsivity by resizing columns

Modern CSS

Edinburgh Napier UNIVERSITY

- Since then, two major native CSS layout system have been popularised
- `flexbox` (Flexible Box) – mainly focuses on rearranging items inside a container depending on the size available
- `grid` – focuses on creating consistent layouts
  - Actually, CSS grids are not a W3C standard yet, but all major browsers have implemented support for it

CanIUse

Edinburgh Napier UNIVERSITY

- The web is a fast-paced technology, there are new frameworks and native features released on the monthly basis
  - It sometimes takes time for browsers to keep-up
- caniuse.com is a really useful website to know how much a certain web “standard” is supported

CSS-Tricks

Edinburgh Napier UNIVERSITY

- Most of these slides have been inspired by content from [css-tricks.com](https://)
- I strongly recommend you give it a read to find more explanations

Edinburgh Napier UNIVERSITY

CSS Flexbox

[css-tricks.com/snippets/css/a-guide-to-flexbox](https://www.css-tricks.com/snippets/css/a-guide-to-flexbox)

CSS Flexbox

Edinburgh Napier UNIVERSITY

- Ideal for a dynamic responsive design, that adapts to the screen space available

- Focuses on a 1-dimensional list of items
  - How one item follows the other
  - Where items are placed within their container
  - How items occupy the space in the container

Container

- First we must declare the display style of the container

```
.container{
    display: flex;
}
```

Container

- Then we specify how items are positioned in the container
  - The direction in which we add them
  - Whether to make new row/column or note when they reach the end of the container

```
.container{
    display: flex;
    flex-direction: row;
    flex-wrap: wrap;
}
```

- Other directions: `column`, `row-reverse`, `column-reverse`
- Other wraps: `nowrap`, `wrap-reverse`

Container

- A shorthand:

```
.container{
    display: flex;
    flex-flow: row wrap;
}
```

Container

- To position items in the direction of the flow:

```
.container{
    display: flex;
    flex-flow: row wrap;
    justify-content: space-evenly;
}
```

- Other values: `flex-start`, `flex-end`, `center`, `space-between`, `space-around`

Container

- To position items in the direction across the flow:

```
.container{
    display: flex;
    flex-flow: row wrap;
    justify-content: space-
evenly;
    align-items: flex-start;
    align-content: space-around;
}
```

- `align-items` concerns the alignment of each individual items
  - Other values: `flex-end`, `center`, `stretch`, `baseline`
- `align-content` concerns the alignment of multiple rows/columns
  - Other values: `flex-start`, `flex-end`, `center`, `stretch`, `space-between`

Items

- By default, items are rendered in the order they are written in the DOM
- However, we can change this

```
.item{
    order: 3;
}
```

- By default, all items have their order set to 0

Items

- We can also control how items expand or shrink if there isn't enough space for them in a row/column

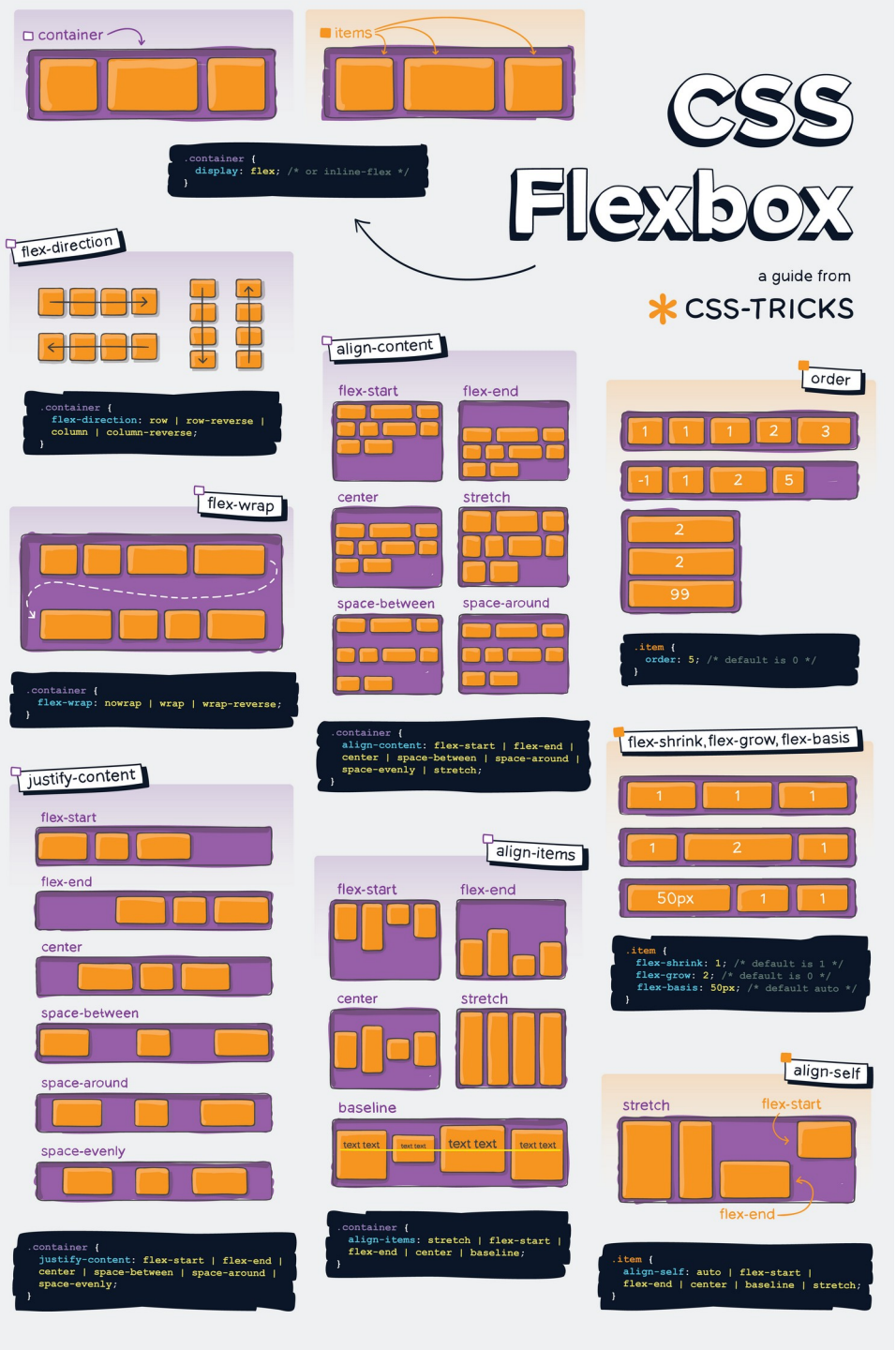
```
.item{
    order: 1;
    flex-grow: 1;
    flex-shrink: 0;
    flex-basis: 300px;
}
```

- `flex-grow`: how an item grows if there is space, default to 0
- `flex-shrink`: how an item shrinks if there isn't enough space, default to 1
- `flex-basis`: what's the targeted size of the item, before growth/shrinkage

Items

- A shorthand:

```
.item{
    order: 1;
    flex: 1 0 300px;
}
```



Edinburgh Napier UNIVERSITY

CSS Flexbox Cheatsheet

CSS-Tricks.com

Edinburgh Napier UNIVERSITY

CSS Grids

[css-tricks.com/snippets/css/complete-guide-grid](https://www.css-tricks.com/snippets/css/complete-guide-grid)

CSS Grids

Edinburgh Napier UNIVERSITY

- Very well suited for dashboards
- Lets you easily set the structure of your page
  - Can be mixed with CSS `@media` queries for more responsiveness
- Focuses on a 2-dimensional grid of items
  - Where items are placed within their container
  - How items occupy the space in the container

Container

- First we must declare the display style of the container

```
.container{
    display: grid;
}
```

Columns

- To set the number and size of columns:

```
.container{
    display: grid;
    grid-template-columns: 1fr 5px 2fr;
}
```

- The `fr` unit refers to the fraction of the available space (after fixed sizes have been subtracted)
- You can use the `repeat(number, value)` CSS function too

Rows

- To set the number and size of rows:

```
.container{
    display: grid;
    grid-template-columns: 1fr 5px 2fr;
    grid-template-rows: 100px repeat(3, 1fr);
}
```

Lines

- We can name lines between columns or rows:

```
.container{
    display: grid;
    grid-template-columns: 1fr [col-1] 5px [col-2] 2fr;
    grid-template-rows: 100px repeat(3, 1fr);
}
```

Areas

- We can then name areas
- Whatever the order in which we declare HTML elements, they will be placed in their named area
- You can also define the start and end lines for each item

```
.container{
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    grid-template-rows: 100px repeat(3, 1fr);
    grid-template-areas:
        "head head head"
        "main main box1"
        "main main box2"
        "desc .    box3";
}

.head{ grid-area: head; }
.main{ grid-area: main; }
.box1{ grid-area: box1; }
.box2{ grid-area: box2; }
.box3{ grid-area: box3; }
.desc{ grid-area: desc; }
```

Placement

- `justify-content` lets you define how rows are placed
  - `start`, `end`, `center`, `stretch`, `space-around`, `space-between`, `space-evenly`
- `align-content` lets you define how columns are placed
  - `start`, `end`, `center`, `stretch`, `space-around`, `space-between`, `space-evenly`
- `place-content` is a shorthand for both `justify-content` and `align-content`

Placement

- `justify-items` lets you define how items are placed horizontally
  - `start`, `end`, `center`, `stretch`
- `align-items` lets you define how items are placed vertically
  - `start`, `end`, `center`, `stretch`, `baseline`
- `place-items` is a shorthand for both `justify-items` and `align-items`
- When inside the rules for items themselves, you can use `justify-self`, `align-self` or `place-self`

Edinburgh Napier UNIVERSITY

CSS Grid Cheatsheet

CSS-Tricks.com

Further Resource

- The website grid.layoutit.com offers a neat interface to write your grid layout

Conclusion

Key Points

- Modern CSS layouts offer great capabilities to render responsive dashboards
- CSS **FlexBox** was created to facilitate 1-dimensional list of items
  - And how these items are placed and sized in dynamic responsive pages
- CSS **Grids** are probably better suited for dashboards
  - They let you define a responsive grid in which elements are placed
  - Mixed with FlexBox or CSS media queries we can have dynamic grids